أشجار البحث (Search Trees)

🎯 مقدمة: من البحث البطيء إلى السريع

المشكلة الأساسية

```text
البحث في مصفوفة عادية:
[15, 3, 28, 9, 41, 7, 22]

للبحث عن 7: يجب فحص كل عنصر واحد تلو الآخر
التعقيد: O(n) - بطيء! 🐢
```

الحل: أشجار البحث

```text
باستخدام BST:
        15
       /  \
      3    28
       \   / \
        9 22  41
       /
      7

للبحث عن 7: 15 → 3 → 9 → 7 (4 خطوات فقط)
التعقيد O(h): حيث h هو الارتفاع - أسرع🚀 !
```

التشبيه البشري🗓️

تخيل مكتبة عملاقة:

❌ بدون نظام: تدور على كل كتاب(O(n))

✅مع نظام:BST

- الكتب الأصغر على اليسار
- الكتب الأكبر على اليمين
- كل خطوة تحذف نصف الخيارات!

🌳 القسم الأول: Binary Search Tree (BST)

التعريف الأساسي

Binary Search Tree = شجرة ثنائية تحقق خاصية الترتيب (Search Property) 1+w3schools

القاعدة الذهبية ⭐

```text
لكل عقدة في الشجرة:

✅ كل العقد في الفرع الأيسر < العقدة
✅ كل العقد في الفرع الأيمن > العقدة

مثال:
        50
       /  \
     30    70
    / \    / \
   20 40  60 80

التحقق:
• 20, 30, 40 < 50 كلها ✅
• 60, 70, 80 > 50 كلها ✅
• 20 < 30 < 40 ✅
• 60 < 70 < 80 ✅
```

التشبيه: "لعبة التخمين

فكر بـ BST كلعبة "خمن الرقم":

"أنا أفكر برقم من 1-100"

- تخمينك: 50

اذا كانت اعلى ← تبحث في الارقام من 51 الى 100

اذا كانت اقل ← تبحث في الارقام من 1 الى 49

BST تعمل بنفس الطريقة!

🔍 خوارزمية البحث (Searching)

الفكرة

```text
نبدأ من الجذر ونقرر:
• ✅ إذا وجدنا العنصر → نرجعه
• ← إذا المطلوب أصغر → نذهب لليسار
• → إذا المطلوب أكبر → نذهب لليمين
• ❌ العنصر غير موجود → إذا وصلنا NULL
```

البحث عن 40 في:

```
        50
       /  \
     30    70
    / \    / \
   20 40  60 80
```

الخطوات:

1. نبدأ من 50: 40 < 50 ← نذهب لليسار
2. نصل إلى 30: 40 > 30 ← نذهب لليمين
3. نصل إلى 40: وجدناه ✅ !

عدد المقارنات: 3 فقط

التعقيد الزمني

```text
Best Case (أفضل حالة):
• العنصر في الجذر: O(1)

Average Case (متوسط):
• شجرة متوازنة: O(log n)

Worst Case (أسوأ حالة):
• شجرة مائلة: O(n)
```

➕ خوارزمية الإدراج (Insertion)

الفكرة

```text
1. نبحث عن المكان المناسب (كأننا نبحث عن العنصر)
2. نضع العنصر الجديد هناك، عندما نصل لـ NULL
3. نحافظ على خاصية BST
```

مثال خطوة بخطوة

```text
إدراج 45 في:

        50
       /  \
     30    70
    / \    / \
   20 40  60 80

الخطوات:
1. 45 < 50 → نذهب لليسار
2. 45 > 30 → نذهب لليمين
3. 45 > 40 → نذهب لليمين
4. نصل إلى NULL → هنا 45 نضع!

النتيجة:
        50
       /  \
     30    70
    / \    / \
   20 40  60 80
       \
        45  ← جديد
```

التعقيد

```text
Insertion: O(h) حيث h = الارتفاع
• شجرة متوازنة: O(log n)
• شجرة مائلة: O(n)
```

🗑️ خوارزمية الحذف (Deletion)

الحالات الثلاث

```text
عند حذف عقدة، أكو 3 حالات:

1. ورقة (Leaf) - بدون أبناء
2. عقدة لها ابن واحد
3. ⚠️ عقدة لها ابنان ← الحالة الصعبة!
```

الحالة 1: حذف ورقة

```text
حذف 20 من:

        50
       /  \
     30    70
    / \
   20 40

بسيط: احذفها مباشرة ↓

        50
       /  \
     30    70
      \
       40
```

الحالة 2: عقدة لها ابن واحد

```text
حذف 30 (لها ابن واحد 40):

        50
       /  \
     30    70
      \
       40

استبدل 30 بـ 40 ↓

        50
       /  \
     40    70
```

الحالة 3: عقدة لها ابنان ⭐ المهمة

```text
حذف 50 (لها ابنان):

        50
       /  \
     30    70
    / \   / \
   20 40 60 80

الحل: "خدعة الـ Inorder Successor"

الخطوات:
1. أوجد أصغر عنصر في الفرع الأيمن (60)
2. انسخ قيمته مكان 50
3. احذف 60 من مكانه الأصلي

↓ النتيجة:

        60  ← 50 استبدلت
       /  \
     30    70
    / \     \
   20 40     80
```

لماذا Inorder Successor؟

```text
Inorder Successor = أصغر عنصر في الفرع الأيمن

السبب:
• ✅ أكبر من كل عناصر الفرع الأيسر
• ✅ أصغر من كل عناصر الفرع الأيمن
• 🎯 يحافظ على خاصية BST تماماً!

بديل آخر:
Inorder Predecessor = أكبر عنصر في الفرع الأيسر
```

مثال كامل للحذف

```text
الشجرة الأصلية:
        50
       /  \
     30    70
    / \   / \
   20 40 60 80

حذف 30 (ابنان):
1. Successor = 40 (أصغر في الفرع الأيمن لـ 30)
2. استبدل 30 بـ 40
3. احذف 40 من مكانه الأصلي

        50
       /  \
     40    70  ← صارت 40 30
    /     / \
   20    60 80
```

المشكلة 💥

```text
الارتفاع h = n (عدد العقد)

البحث: O(n) - نفس المصفوفة! 🐢
الإدراج: O(n)
الحذف: O(n)

فقدنا كل ميزة BST! ☠️
```

الحل: الأشجار المتوازنة

```text
نحتاج شجرة "تتوازن نفسها تلقائياً":

1. AVL Tree 🎯 (دقيقة وصارمة)
2. Red-Black Tree 🎨 (مرنة وسريعة)
```

⚖️ القسم الثاني: AVL Trees

التعريف

AVL Tree = Binary Search Tree + شرط التوازن الصارم

> سميت على اسم مخترعيها: Adelson-Velsky و Landis (1962)

شرط التوازن (Balance Condition)

```text
Balance Factor (BF) = Height(Left) - Height(Right)

لكل عقدة:
BF ∈ {-1, 0, 1}

يعني:
✅ BF = 0  → متوازنة تماماً
✅ BF = 1  → اليسار أعلى بواحد
✅ BF = -1 → اليمين أعلى بواحد
❌ BF ≥ 2 أو ≤ -2 → غير متوازنة! يجب التصحيح
```

مثال حساب BF

```text
       30(h=2)
      /       \
   20(h=1)   40(h=0)
   /
10(h=0)

حساب BF لكل عقدة:
• 10 العقدة: BF = 0 - 0 = 0 ✅
• 20 العقدة: BF = 1 - 0 = 1 ✅
• 40 العقدة: BF = 0 - 0 = 0 ✅
• 30 العقدة: BF = 2 - 1 = 1 ✅

الشجرة متوازنة!
```

🔄 الدورانات (Rotations)

لماذا الدورانات؟

```text
عند إدراج/حذف عنصر، قد يختل التوازن:

       30
      /
    20
   /
  10

BF(30) = 2 ❌ غير متوازن!

الحل: دوران → إعادة الهيكلة
```

أنواع الدورانات الأربعة

```text
1. Right Rotation (LL Case)
2. Left Rotation (RR Case)
3. Left-Right Rotation (LR Case)
4. Right-Left Rotation (RL Case)
```

1. Right Rotation (LL Case)

```text
الحالة: خلل في اليسار-اليسار (Left-Left)

قبل:
      z (BF=2)
     /
    y (BF=1)
   /
  x

بعد الدوران لليمين:
      y
     / \
    x   z

الخطوات:
1. y تصبح الجذر الجديد
2. z تنزل لليمين
3. الفرع الأيمن لـ y يصبح الفرع الأيسر لـ z
```

مثال رقمي

```text
قبل:
      30 (BF=2) ← خلل
     /
    20 (BF=1)
   /
  10

بعد rightRotate(30):
      20
     /  \
    10   30

الآن متوازنة! ✅
```

2. Left Rotation (RR Case)

```text
الحالة: خلل في اليمين-اليمين (Right-Right)

قبل:
  z (BF=-2)
   \
    y (BF=-1)
     \
      x

بعد الدوران لليسار:
      y
     / \
    z   x

الخطوات:
1. y تصبح الجذر الجديد
2. z تنزل لليسار
3. z يصبح الفرع الأيمن لـ y الفرع الأيسر لـ
```

مثال رقمي

```text
قبل:
10 (BF=-2) ← خلل
  \
   20 (BF=-1)
     \
      30

بعد leftRotate(10):
      20
     /  \
    10   30

متوازنة! ✅
```

3. Left-Right Rotation (LR Case) `codingeek`

```text
الحالة: خلل في اليسار-اليمين (Left-Right)

قبل:
    z (BF=2)
   /
  y (BF=-1)
   \
    x

الحل: دورانان!
1. Left Rotation على y
2. Right Rotation على z

بعد:
      x
     / \
    y   z
```

مثال رقمي خطوة بخطوة

```text
قبل:
    30 (BF=2)
   /
  10 (BF=-1)
    \
     20

الخطوة 1: leftRotate(10)
    30
   /
  20
 /
10

الخطوة 2: rightRotate(30)
      20
     /  \
    10   30

متوازنة! ✅
```

4. Right-Left Rotation (RL Case)

```text
الحالة: خلل في اليمين-اليسار (Right-Left)

قبل:
  z (BF=-2)
   \
    y (BF=1)
   /
  x

الحل: دورانان!
1. Right Rotation على y
2. Left Rotation على z

بعد:
      x
     / \
    z   y
```

مثال رقمي

```text
قبل:
  10 (BF=-2)
    \
     30 (BF=1)
    /
   20

1 الخطوة: rightRotate(30)
  10
    \
     20
       \
        30

2 الخطوة: leftRotate(10)
      20
     /  \
    10   30

متوازنة! ✅
```

📊 جدول تحديد نوع الدوران

الدوران المطلوب	الحالة	BF للابن	BF للأب
Right Rotation	LL	1+ أو 0	2+
Left Rotation	RR	1- أو 0	2-
Left-Right	LR	1-	2+
Right-Left	RL	1+	2-

🎨 القسم الثالث: Red-Black Trees

التعريف

BST = **Red-Black Tree** + نظام تلوين + قواعد توازن

الفكرة الأساسية

```text
بدل حساب الارتفاع (مثل AVL):
→ نستخدم "ألوان" (أحمر/أسود)
→ نطبق قواعد للألوان
→ هذه القواعد تضمن التوازن!
```

التشبيه 🚦

```text
تخيل إشارات المرور:

🔴 "أحمر" = "عقدة حديثة الإدراج
⚫ "أسود" = "عقدة مستقرة

القواعد مثل قوانين المرور:
• ممنوع إشارتين حمراويتين متتاليتين
• كل طريق من الجذر للورقة يمر بنفس عدد الإشارات السوداء
```

القواعد الخمسة

```text
1. Root Property:
   الجذر دائماً أسود ⚫

2. External Property:
   كل العقد الخارجية (NULL) سوداء ⚫

3. Red Property:
   ممنوع أحمر → أحمر (لا يمكن لعقدة حمراء أن يكون لها ابن أحمر)
   🔴 → 🔴 ❌

4. Black-Height Property:
   كل مسار من الجذر إلى ورقة يمر بنفس عدد العقد السوداء

5. Node Color:
   كل عقدة إما حمراء 🔴 أو سوداء ⚫
```

مثال توضيحي

```text
Red-Black Tree صحيحة:

        ⚫30
       /    \
    🔴20     ⚫40
    /  \       \
⚫10   ⚫25    🔴50

التحقق من القواعد:
✅ الجذر (30) أسود
✅ لا يوجد أحمر متتالي
✅ Black-Height من الجذر:
   • 30→20→10: 2 (10 ,30) أسود
   • 30→20→25: 2 (25 ,30) أسود
   • 30→40→50: 2 (40 ,30) أسود
   كلها متساوية ✅
```

مثال خاطئ

```text
❌ Red-Black Tree خاطئة:

        🔴30  ← الجذر أحمر (خرق القاعدة 1)
       /    \
    🔴20     ⚫40  ← أحمر متتالي (خرق القاعدة 3) 30→20
    /
⚫10
```

عمليات التصحيح

```text
عند الإدراج/الحذف، قد نحتاج:
1. Recoloring: تغيير الألوان
2. Rotations: دورانات (مثل AVL)
```

مثال الإدراج

```text
إدراج 15 في:

        ●20
      /    \
   ●10     ●30

الخطوات:
1. أدرج 15 كعقدة حمراء (القاعدة)
        ●20
      /    \
   ●10     ●30
     \
    🔴15

2. تحقق من القواعد:
   • 10 لا مشكلة ← أحمر 15 أسود، ✅!

النتيجة النهائية:
        ●20
      /    \
   ●10     ●30
     \
    🔴15
```

متى نستخدم أيهم؟

```text
BST العادية:
✅ البيانات عشوائية ومتوازنة طبيعياً
✅ تطبيقات بسيطة
❌ بيانات مرتبة أو شبه مرتبة

AVL:
✅ تطبيقات تحتاج بحث سريع جداً
✅ قراءة أكثر من كتابة (مثل: قواعد بيانات للقراءة)
❌ تطبيقات فيها إدراج/حذف كثير

Red-Black:
✅ تطبيقات متوازنة (قراءة وكتابة)
✅ تحديثات كثيرة (مثل: std::map في C++)
✅ استخدام عام في أنظمة التشغيل
```

📊 مقارنة شاملة: BST vs AVL vs Red-Black

الخاصية	BST عادية	AVL	Red-Black
التوازن	لا يوجد	صارم جداً	مرن
البحث	O(h)، أسوأ O(n)	O(log n) دائماً	O(log n) دائماً
الإدراج	O(h)	O(log n) + دورانات	O(log n) + قليل دورانات
الحذف	O(h)	O(log n) + دورانات	O(log n) + قليل دورانات
الارتفاع	قد يصل لـ n	≈ 1.44 log n	≈ 2 log n
تعقيد التنفيذ	بسيط	متوسط	معقد
الاستخدام	بيانات عشوائية	قراءة كثيرة	تحديث كثير
ذاكرة إضافية	لا شيء	height في كل عقدة	color في كل عقدة
عدد الدورانات (إدراج)	0	1-2	0-2
عدد الدورانات (حذف)	0	log n-1	0-3

📝 ملخص سريع للمراجعة

BST

```text
• خاصية البحث: يسار < جذر < يمين
• البحث: O(h)
• الإدراج: O(h)
• الحذف: O(h) + Inorder Successor
• المشكلة: Skewed Tree → O(n)
```

AVL

```text
• Balance Factor: |h_left - h_right| ≤ 1
• 4 دورانات: LL, RR, LR, RL
• التعقيد: O(log n) مضمون
• مناسب للقراءة الكثيرة
```

Red-Black

```text
• 5 قواعد للألوان
• الجذر أسود دائماً
• ممنوع أحمر متتالي
• Black-Height متساوي
• مناسب للتحديث الكثير
```

True/False

```text
1. كل AVL Tree هي BST.
   ✅ True (AVL تضيف شرط التوازن على BST)

2. كل BST هي AVL Tree.
   ❌ False (BST قد تكون غير متوازنة)

3. Skewed Tree لها نفس أداء المصفوفة.
   ✅ True (كلاهما O(n) للبحث)

4. Red-Black أسرع من AVL في الإدراج.
   ✅ True (دورانات أقل)

5. في AVL، يمكن أن يكون BF = 2.
   ❌ False (يجب إعادة التوازن فوراً)
```

Tracing

السؤال: أدرج 15, 10, 20, 8, 12, 25 في AVL Tree خطوة بخطوة.

الحل:

```text
الخطوة 1: إدراج 15
15

الخطوة 2: إدراج 10
    15
   /
  10

الخطوة 3: إدراج 20
    15
   /  \
  10   20

الخطوة 4: إدراج 8
      15
     /  \
    10   20
   /
  8

BF(10) = 1, BF(15) = 1 ← متوازنة ✅

الخطوة 5: إدراج 12
      15
     /  \
    10   20
   / \
  8  12

الخطوة 6: إدراج 25
      15
     /  \
    10   20
   / \     \
  8  12    25

✅ كل العقد متوازنة! النتيجة النهائية
```